Основные архитектуры CNN

&&

Харитон Пицик

2026-04-15

Клуб машинного обучения

Ранее мы уже говорили, что типичный пайплайн CNN выглядит следующим образом:

$$\text{Conv} \to \text{ReLU} \to \text{Pooling} \to \ldots \to \text{Flatten} \to \text{Linear} \to \text{Softmax}$$

Однако такого пайплайна, оказывается, недостаточно.

Recap: Почему CNN вообще работает? &&

Однако почему CNN вообще даёт какие-либо результаты?

Recap: Почему CNN вообще работает? &&

Однако почему CNN вообще даёт какие-либо результаты?

1. Локальная обработка, пиксели влияют друг на друга;

Recap: Почему CNN вообще работает? &&

Однако почему CNN вообще даёт какие-либо результаты?

1. Локальная обработка, пиксели влияют друг на друга;
2. Shared weights;

Recap: Почему CNN вообще работает? &&

Однако почему CNN вообще даёт какие-либо результаты?

1. Локальная обработка, пиксели влияют друг на друга;
2. Shared weights;
3. В зависимости от глубины, фильтры выучивают разные признаки.

Recap: Проблемы CNN

&&

Recap: Проблемы CNN

&&

1. Потеря информации при пулинге;

Recap: Проблемы CNN

&&

1. Потеря информации при пулинге;
2. Невозможность обучать глубокие сети;

Recap: Проблемы CNN

&&

1. Потеря информации при пулинге;
2. Невозможность обучать глубокие сети;
3. Невозможность передать далее какие-то ранее полученные признаки.

Проблема неглубокости CNN

&&

Первая проблема, которую начали решать – невозможность обучать глубокие сети. Решают её с помощью:

Проблема неглубокости CNN

&&

Первая проблема, которую начали решать – невозможность обучать глубокие сети. Решают её с помощью:

1. ReLU

Проблема неглубокости CNN

&&

Первая проблема, которую начали решать – невозможность обучать глубокие сети. Решают её с помощью:

1. ReLU
2. Dropout

Проблема неглубокости CNN

&&

Первая проблема, которую начали решать – невозможность обучать глубокие сети. Решают её с помощью:

1. ReLU
2. Dropout
3. Data Augmentation

Проблема неглубокости CNN

&&

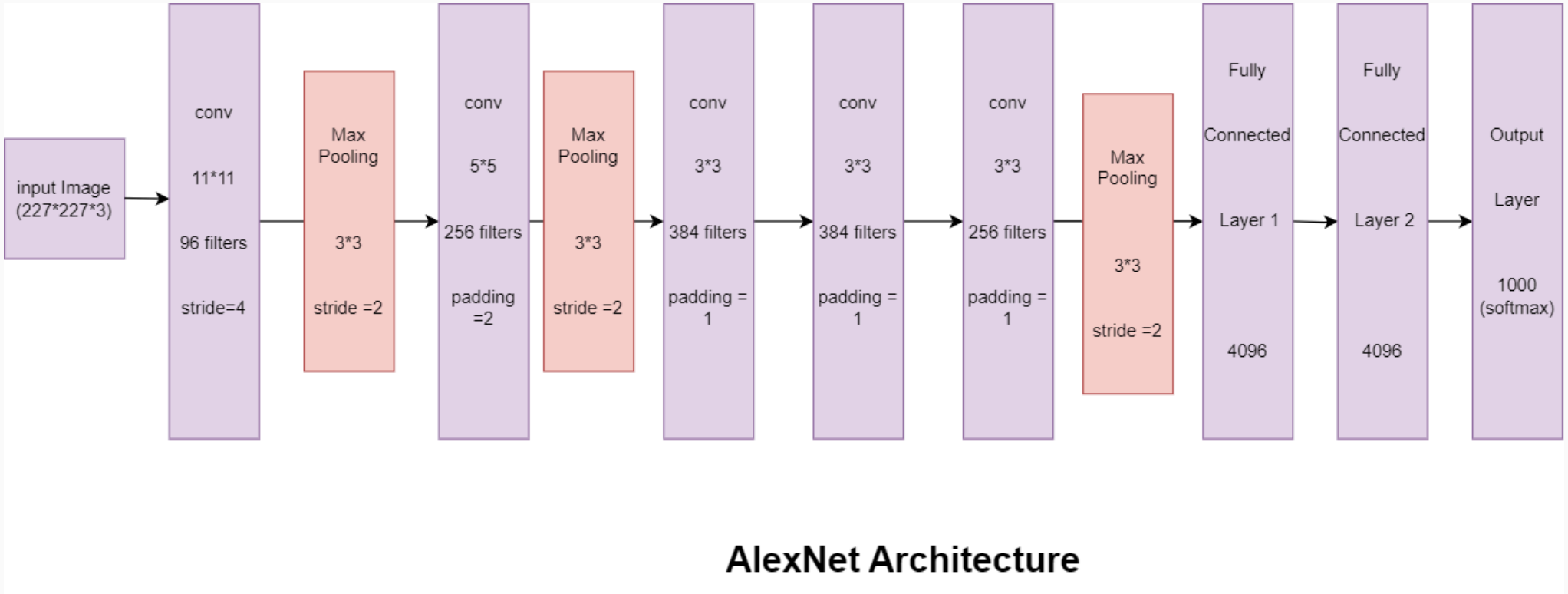
Первая проблема, которую начали решать – невозможность обучать глубокие сети. Решают её с помощью:

1. ReLU
2. Dropout
3. Data Augmentation

Комбинируя всё вместе получается архитектура, которую называют AlexNet.

Проблема неглубокости CNN

&&



Создание AlexNet

&&

AlexNet была реализована в 2012 году Алексом Крижевским, Ильей Суцкевером и Джеффри Хинтоном. Первая реализация была написана на C++ и обучена на двух GTX 580.

Создание AlexNet

&&

AlexNet была реализована в 2012 году Алексом Крижевским, Ильей Суцкевером и Джеффри Хинтоном. Первая реализация была написана на C++ и обучена на двух GTX 580.

Данная архитектура (а точнее ансамбль из 7 моделей такой архитектуры) побеждала в соревновании ImageNet в 2012 году.

Ограничения AlexNet

&&

Ограничения AlexNet

&&

- огромные FC слои

Ограничения AlexNet

&&

- огромные FC слои
- архитектура “грязная”

Ограничения AlexNet

&&

- огромные FC слои
- архитектура “грязная”
- нет чёткого принципа масштабирования

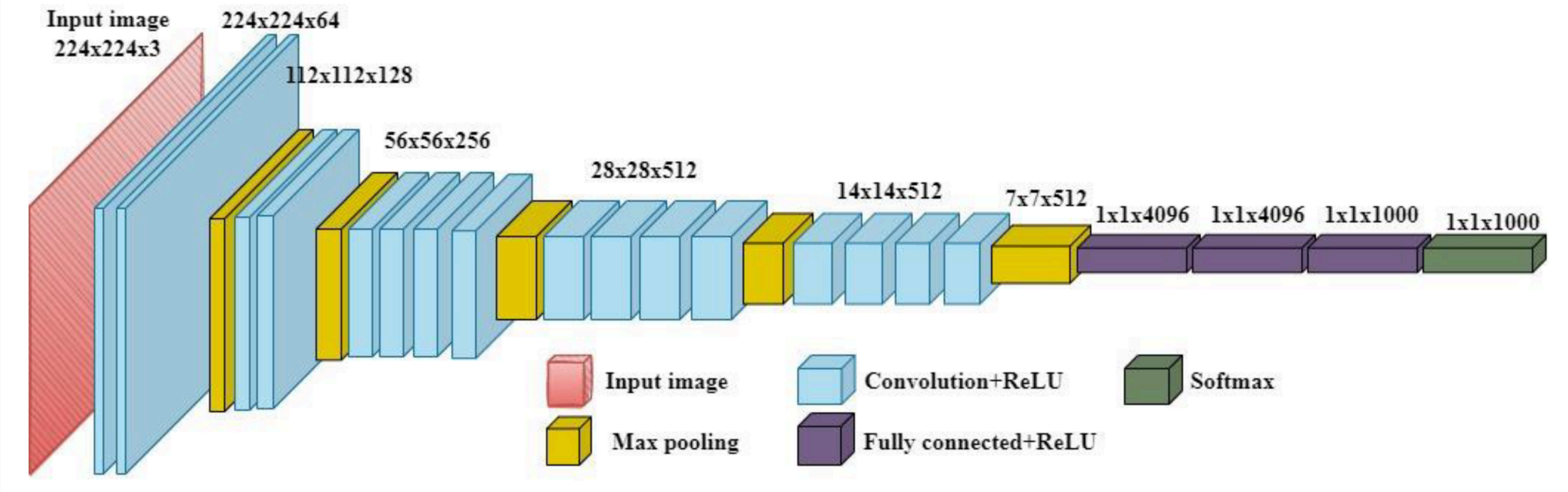
Можно ли как-то проще?

&&

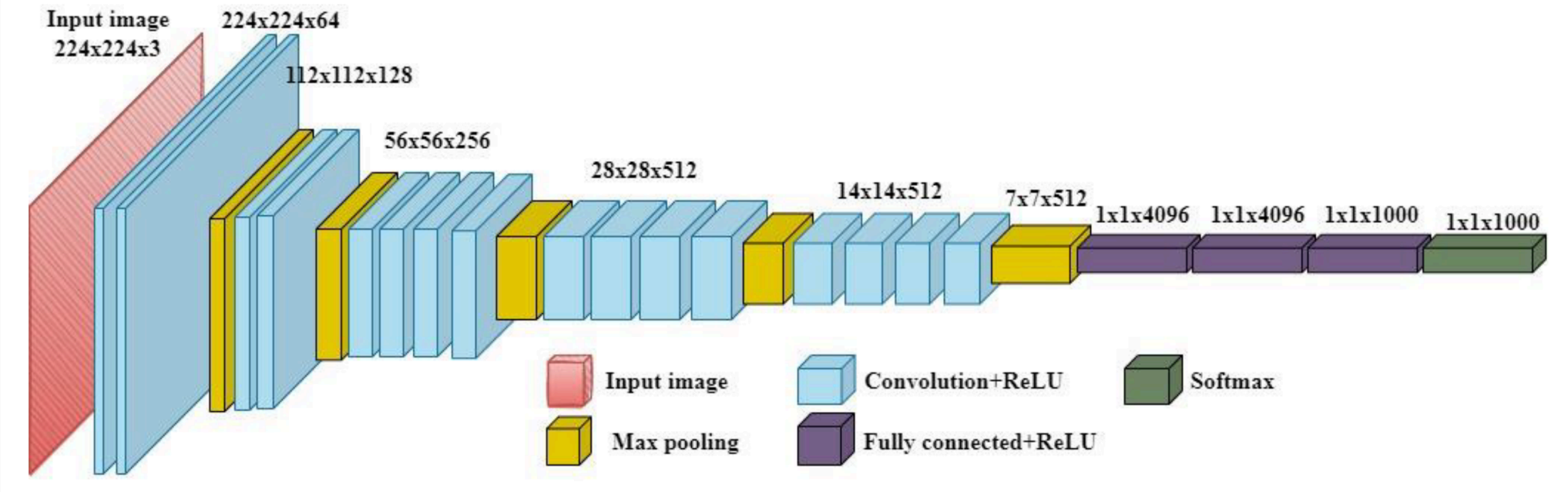
Что, если мы не будем как-то изворачиваться, а будем делать однотипную архитектуру, но глубже?

Можно ли как-то проще?

&&



Можно ли как-то проще? &&



Такая архитектура называется VGG. Конкретно на слайде – архитектура VGG-19.

Создание VGG

&&

VGG была предложена группой Visual Geometry Group (Оксфорд) в 2014 году. Основная идея: сделать архитектуру максимально простой и системной, но при этом глубокой. Для этого использовались только 3×3 свёртки.

Создание VGG

&&

VGG была предложена группой Visual Geometry Group (Оксфорд) в 2014 году. Основная идея: сделать архитектуру максимально простой и системной, но при этом глубокой. Для этого использовались только 3×3 свёртки.

В отличие от AlexNet:

Создание VGG

&&

VGG была предложена группой Visual Geometry Group (Оксфорд) в 2014 году. Основная идея: сделать архитектуру максимально простой и системной, но при этом глубокой. Для этого использовались только 3×3 свёртки.

В отличие от AlexNet:

- вводим единый строительный блок

Создание VGG

&&

VGG была предложена группой Visual Geometry Group (Оксфорд) в 2014 году. Основная идея: сделать архитектуру максимально простой и системной, но при этом глубокой. Для этого использовались только 3×3 свёртки.

В отличие от AlexNet:

- вводим единый строительный блок
- увеличиваем глубину сети

Архитектура VGG

&&

Сеть строится из повторяющихся блоков:

Архитектура VGG

&&

Сеть строится из повторяющихся блоков:

[Conv 3×3 $\longrightarrow$ ReLU] × N $\longrightarrow$ Pooling

Далее увеличивается количество каналов, а размер изображения уменьшается.

Архитектура VGG

&&

Сеть строится из повторяющихся блоков:

[Conv 3×3 $\longrightarrow$ ReLU] × N $\longrightarrow$ Pooling

Далее увеличивается количество каналов, а размер изображения уменьшается.

Типичная схема каналов: 64 $\longrightarrow$ 128 $\longrightarrow$ 256 $\longrightarrow$ 512 $\longrightarrow$ 512

Архитектура VGG

&&

Сеть строится из повторяющихся блоков:

[Conv 3×3 $\longrightarrow$ ReLU] × N $\longrightarrow$ Pooling

Далее увеличивается количество каналов, а размер изображения уменьшается.

Типичная схема каналов: 64 $\longrightarrow$ 128 $\longrightarrow$ 256 $\longrightarrow$ 512 $\longrightarrow$ 512

В конце: Flatten $\longrightarrow$ Fully Connected $\longrightarrow$ Softmax

Receptive field

&&

Receptive field — это область входного изображения, которая влияет на значение конкретного нейрона.

Receptive field

&&

Receptive field — это область входного изображения, которая влияет на значение конкретного нейрона.

Пример:

- $1 \times$ Conv 3×3 $\longrightarrow$ receptive field = 3×3
- $2 \times$ Conv 3×3 $\longrightarrow$ receptive field = 5×5
- $3 \times$ Conv 3×3 $\longrightarrow$ receptive field = 7×7

Receptive field

&&

Receptive field — это область входного изображения, которая влияет на значение конкретного нейрона.

Пример:

- $1 \times$ Conv 3×3 $\longrightarrow$ receptive field = 3×3
- $2 \times$ Conv 3×3 $\longrightarrow$ receptive field = 5×5
- $3 \times$ Conv 3×3 $\longrightarrow$ receptive field = 7×7

Каждый слой увеличивает receptive field, потому что комбинирует информацию из предыдущего слоя.

Рост происходит за счёт двух вещей:

Как он растёт

&&

Рост происходит за счёт двух вещей:

1. Свёртки

Как он растёт

&&

Рост происходит за счёт двух вещей:

1. Свёртки
2. Pooling

Проблемы VGG

&&

Проблемы VGG

&&

1. Огромное количество параметров;

Проблемы VGG

&&

1. Огромное количество параметров;
2. Медленная;

Проблемы VGG

&&

1. Огромное количество параметров;
2. Медленная;
3. Плохо масштабируема по глубине.

Проблема постановки задачи

&&

Сейчас наша задача выглядит как аппроксимация некоторой функции $f(x)$, однако чем больше у нас весов, тем сложнее на самом деле это сделать.

Проблема постановки задачи &&

Сейчас наша задача выглядит как аппроксимация некоторой функции $f(x)$, однако чем больше у нас весов, тем сложнее на самом деле это сделать.

Основное предположение заключается в том, что было бы проще учить *остатки*, т.е.

$$y = f(x) + x$$

где x - это входы слоя.

Пробрасываем слой

&&

$$y = f(x) + x$$

Получается, что таким образом мы **пробрасываем** x через какой-то слой.

Пробрасываем слой

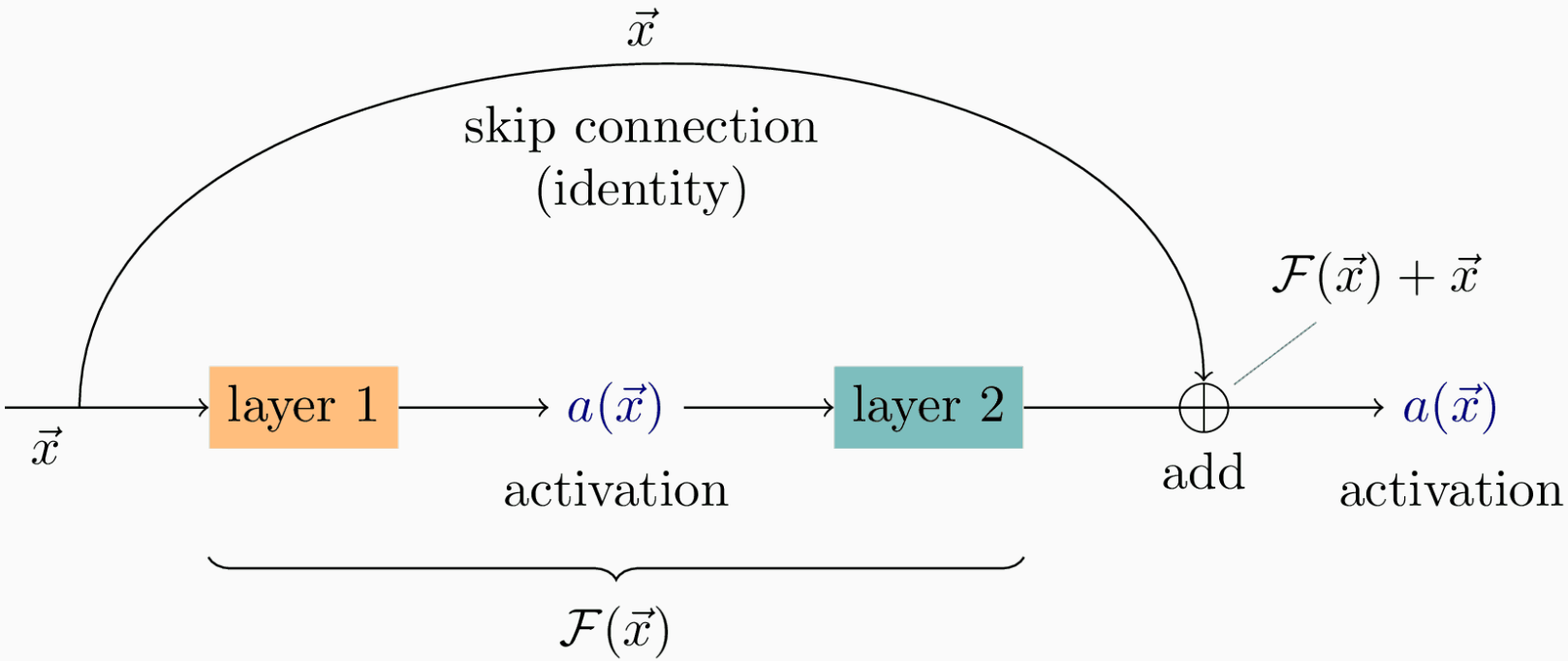
$$y = f(x) + x$$

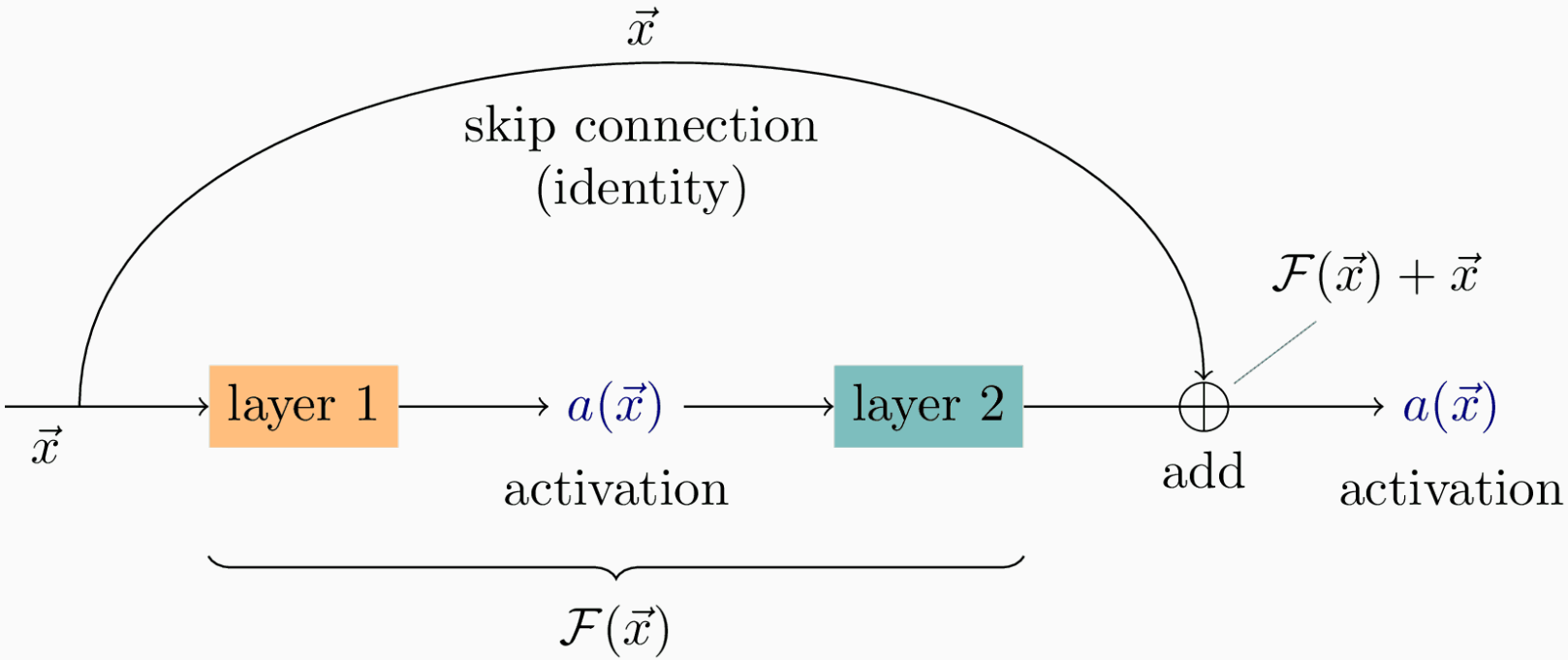
Получается, что таким образом мы **пробрасываем** x через какой-то слой.

Такой приём называется **skip connections**.

Skip connections

&&





Теперь модель учит не преобразование над x, а что *нужно добавить* к x.

Почему это хорошо?

Почему это хорошо?

&&

1. Решаем проблему затухающего градиента;

Почему это хорошо?

&&

1. Решаем проблему затухающего градиента;
2. Сети с skip connections сходятся быстрее.

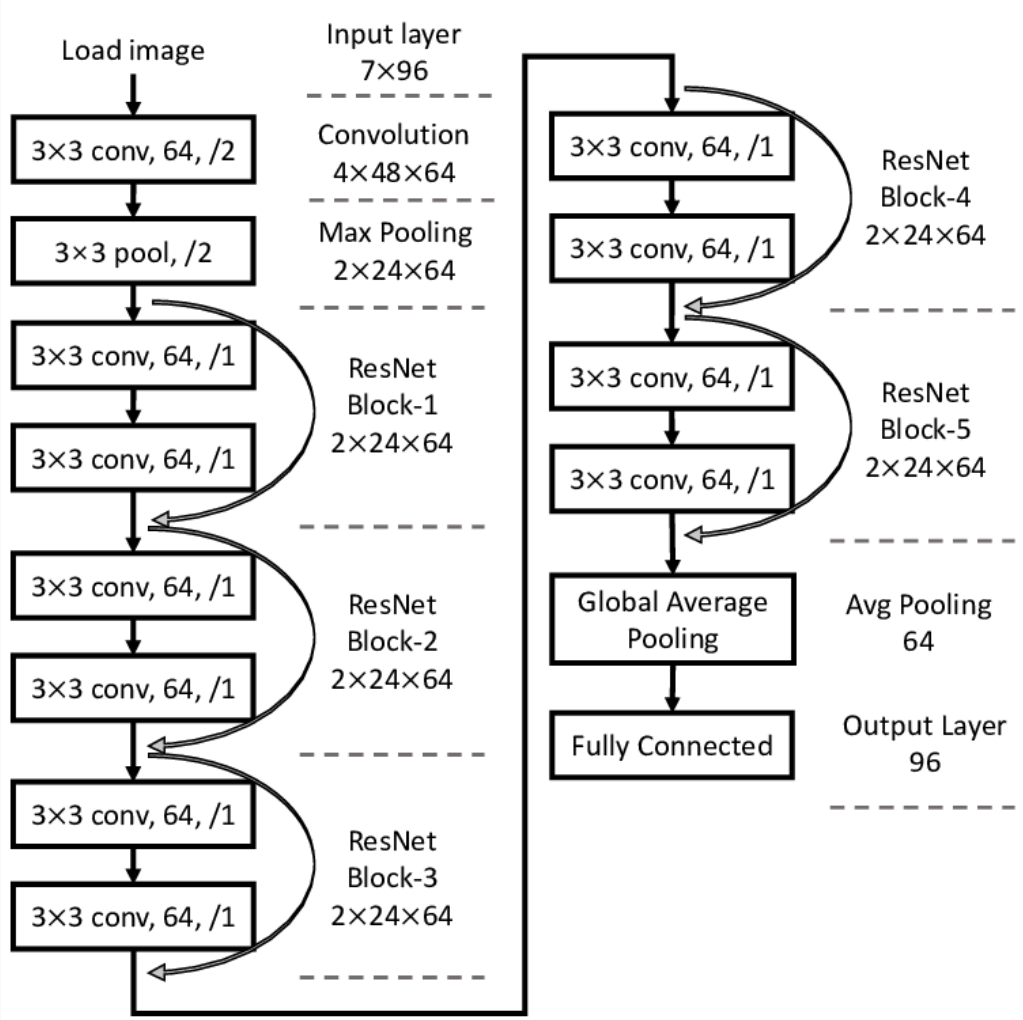
Архитектура с skip connections. &&

Впервые такой подход в обучении CNN был представлен в архитектуре ResNet. В зависимости от количества слоёв, бывают:

- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101
- ResNet-152

Друг от друга они отличаются только количеством блоков, однако условно делятся на “до 50” и “после 50”.

Архитектура с skip connections. &&



Архитектура с skip connections. &&

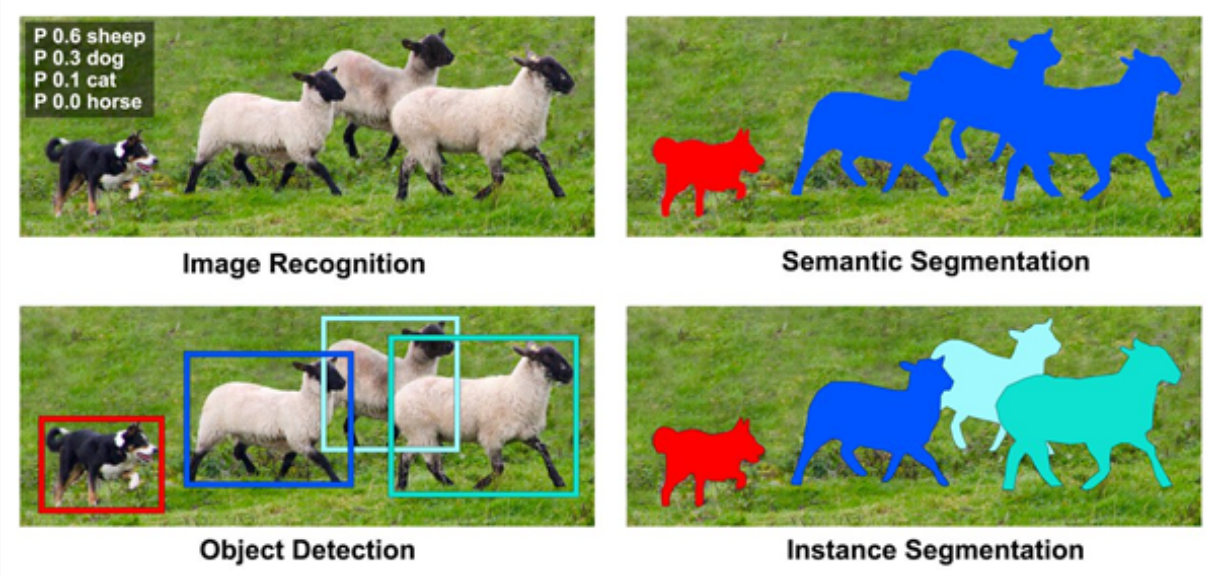
Такое архитектурное решение принесло победу на соревновании ImageNet 2015 года, а также позволило оптимизировать глубокие сети и иначе взглянуть на задачу оптимизации.

На данный момент практически все современные архитектуры содержат в себе приём skip connections.

Задача сегментации

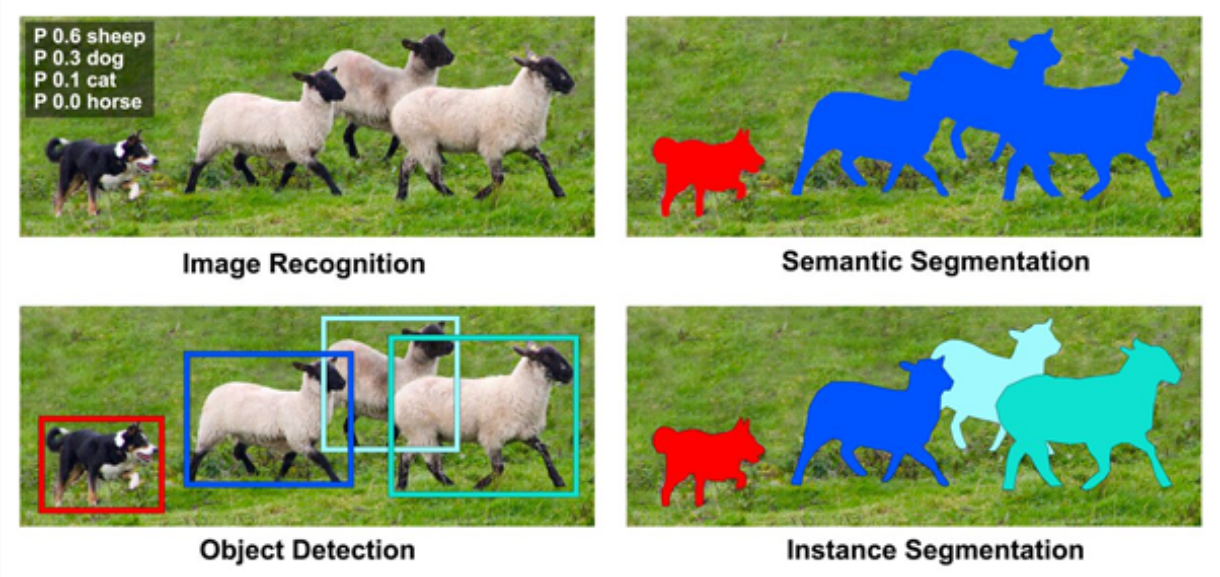
Задача сегментации

&&



Задача сегментации

&&



Задача сегментации заключается в том, что каждый пискель изображения должен получить метку класса.

Архитектурное решение задачи

&&

Архитектура состоит из двух частей:

- Encoder – как обычный CNN
- Decoder – восстанавливает изображение

Повторяющийся блок:

$$\text{Conv} \to \text{ReLU} \to \text{Conv} \to \text{ReLU} \to \text{Pooling}$$

Encoder &&

Повторяющийся блок:

$$\text{Conv} \to \text{ReLU} \to \text{Conv} \to \text{ReLU} \to \text{Pooling}$$

- уменьшается размер изображения
- увеличивается число каналов

Bottleneck &&

Самая глубокая часть сети:

$$\text{Conv} \to \text{Conv}$$

Bottleneck &&

Самая глубокая часть сети:

$$\text{Conv} \to \text{Conv}$$

- максимальный receptive field => максимальная абстракция

Decoder

&&

Основной блок:

$$\text{Upsample} \to \text{Conv} \to \text{Conv}$$

Decoder &&

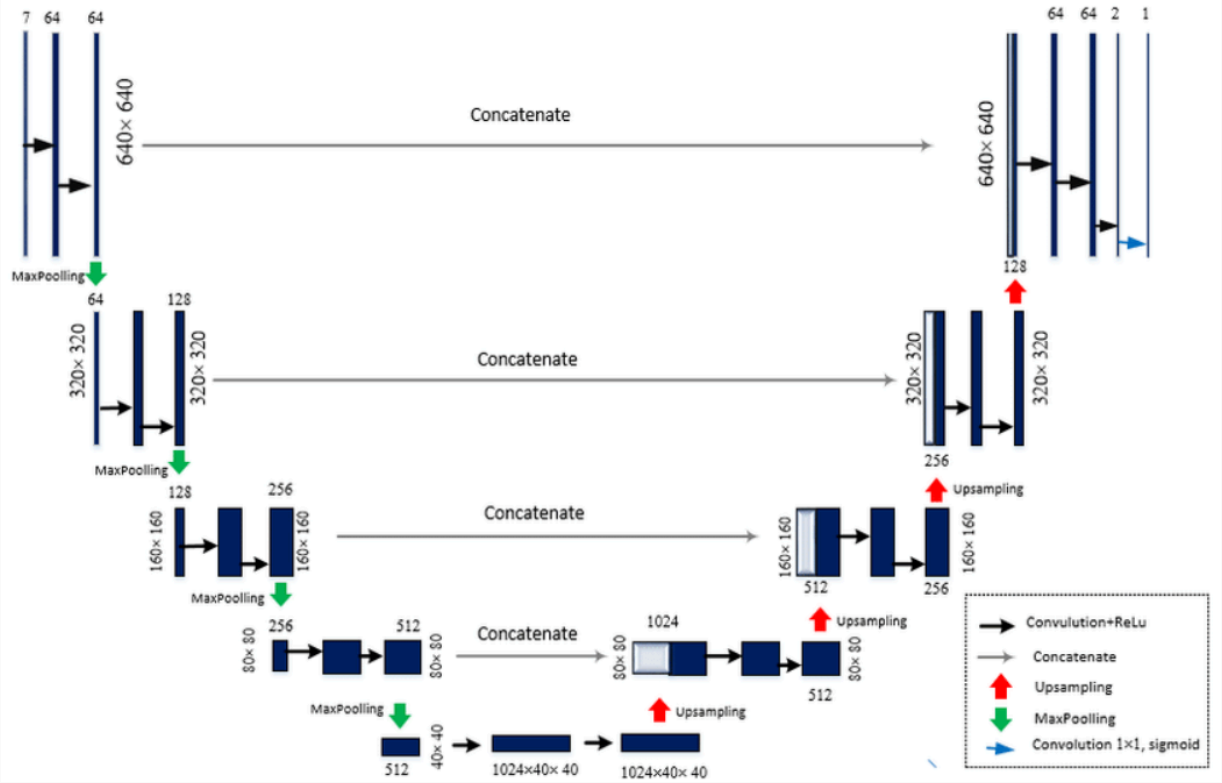
Основной блок:

$$\text{Upsample} \to \text{Conv} \to \text{Conv}$$

- уменьшаем число каналов
- постепенно возвращаем размерность к исходной

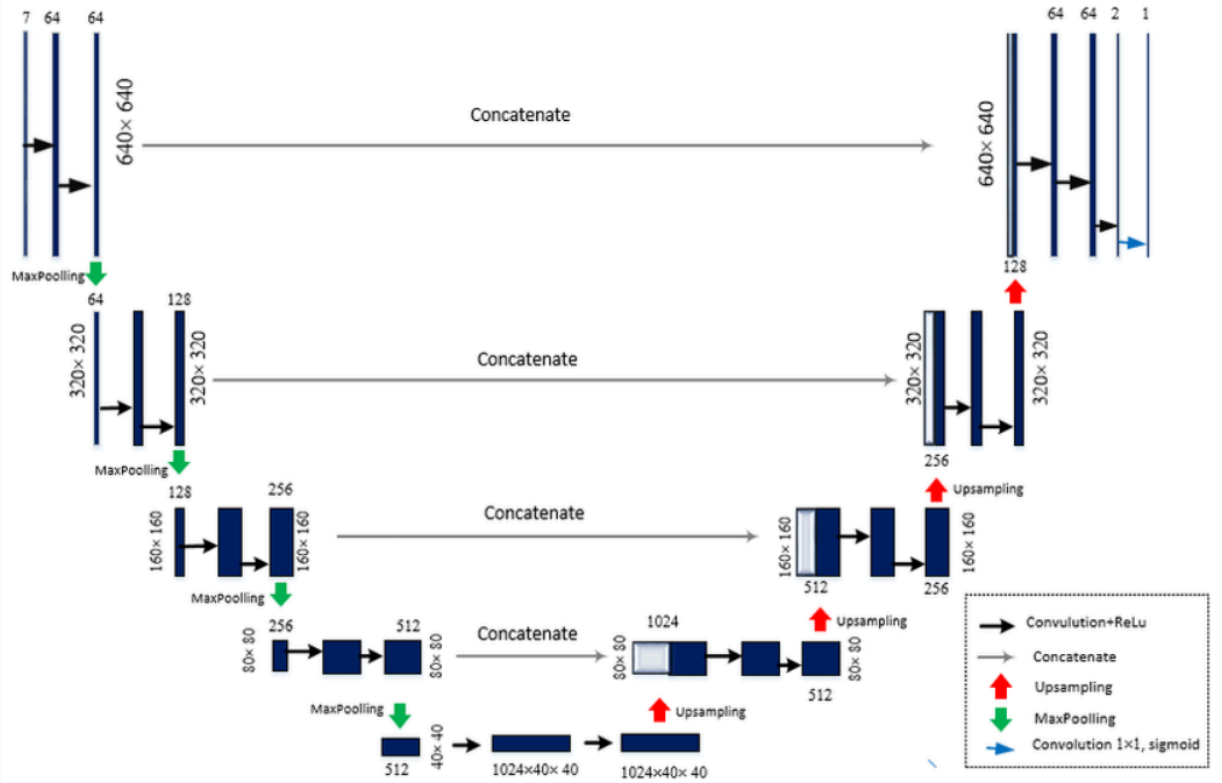
Decoder

&&


7 64 64
640× 640
Concatenate
64 64 2 1
640× 640
MaxPoolling
64
128
320× 320
320× 320
Concatenate
320× 320
320× 320
128
MaxPoolling
128
256
160× 160
160× 160
Concatenate
160× 160
160× 160
512
256
Upsampling
MaxPoolling
256
512
80× 80
80× 80
Concatenate
1024
80× 80
80× 80
512
Upsampling
Upsampling
MaxPoolling
512
40× 40
1024×40× 40
1024×40× 40
Convulution+ReLu
Concatenate
Upsampling
MaxPoolling
Convolution 1×1, sigmoid

Decoder

&&



Архитектура похожа букву U, поэтому UNet.

Вернёмся к задаче сегментации

&&

Раз мы начали говорить о сегментации как о задаче машинного обучения, стоит рассмотреть основные функции потерь в рамках этой задачи.

Dice Loss

&&

Dice loss выражает сходство предсказанной и целевой областей:

Dice Loss

&&

Dice loss выражает сходство предсказанной и целевой областей:

$$\text{Dice coef} = \frac{2 * |A \cap B|}{|A| + |B|}$$

$$\text{Dice loss} = 1 - \text{Dice coef}$$

IoU является более строгой функцией, которая штрафует ошибки.

IoU является более строгой функцией, которая штрафует ошибки.

$$\text{IoU Loss} = 1 - \frac{|A \cap B|}{|A \cup B|}$$

$$\text{Dice} = \frac{2\ \text{IoU}}{\text{IoU} + 1}$$